Akash Singh

SRN-PES2UG24CS042

Part 1-Requirements Table (5 FRs and 2 NFRs)

Req ID	Type	Description	Priority	Acceptance Criteria (pass / fail)	Rationale
FR-001	Functional	System should receive orders from Shopify or WooCommerce, check whether the products are available from the supplier, and send a purchase order to the supplier	High	Pass: A purchase order is created with the correct customer delivery address within 5 seconds of receiving the order. Fail: A purchase order is sent for a product that the supplier has marked as out of stock.	This is the main order flow of the system.
FR-002	Functional	System shall split an order that has items from more than one supplier into one sub-order for each supplier	High	Pass: An order with items from 2 suppliers creates 2 suborders, and both are linked to the same parent order ID. Fail: Only one sub-order is created, or an item is missing from the sub-orders.	A supplierr can ship only the items that it stocks.
FR-003	Functional	System shall put the sub-order on hold and send an alert to the E-Store Owner when the supplier API returns a stock count of zero for an item	High	Pass: A stock reply of 0 sets the sub-order status to On Hold and the alert reaches the owner within 1 minute. Fail: The PO is still sent, or no alert is sent.	Stops the store from selling an item that cannot be shipped.
FR-004	Functional	System shall save the tracking number and courier name sent by the supplier and shall write them back to the matching storefront order	Medium	Pass: The tracking number on the storefront order page is the same as the one sent by the supplier. Fail: The field stays empty, or it holds the tracking number of a different order.	The owner and the customer need to know where the parcel is.
FR-005	Functional	System shall show a dashboard that lists every order with its sub-orders, supplier name, current status, and tracking number	Medium	Pass: The dashboard lists all orders of the last 30 days with the correct status. Fail: An order is misssing, or its status is older than the last update.	The owner needs one screen to follow all orders.

Req ID	Type	Description	Priority	Acceptance Criteria (pass / fail)	Rationale
NFR-001	Nonfunctional (Performance and Security)	The system shall reply to an inbound storefront webhook with HTTP 200 within 200 ms, and shall accept the webhook only after the HMAC signature is verified	High	Pass: A load test at 100 webhooks per second shows a reply time under 200 ms, and every request with a wrong signature is rejected. Fail: The reply takes more than 200 ms, or an unsigned request is accepted.	A slow reply makes the storefront send the same webhook again and again.
NFR-002	Nonfunctional (Reliability)	If a supplier API call fails, the system should try again 3 times, waiting 30 seconds between each attempt. The order should stay in the queue until the request is successfully processed.	High	Pass: When the supplier API is unavailable, the system makes 3 retry attempts, records them in the log, and keeps the order in the queue. Fail: The order is removed from the queue or fewer than 3 retry attempts are made.	Supplier APIs go down often and no order should be lost.

1. Actors and Use Cases Extracted from the Table

Actors (4):

1. E-Store Owner - owns the store, watches orders, and fixes items that are on hold.
2. Supplier Partner - gives stock counts, accepts purchase orders, and returns tracking numbers.
3. Storefront Platform (Shopify / WooCommerce) - sends the order webhook to the system.
4. Courier Service - gives the shipment status for a tracking number.

Use Cases (8):

UC-01 Process Storefront Order (from FR-001)

UC-02 Check Supplier Stock (from FR-001)

UC-03 Split Multi-Vendor Order (from FR-002)

UC-04 Send Supplier Purchase Order (from FR-001)

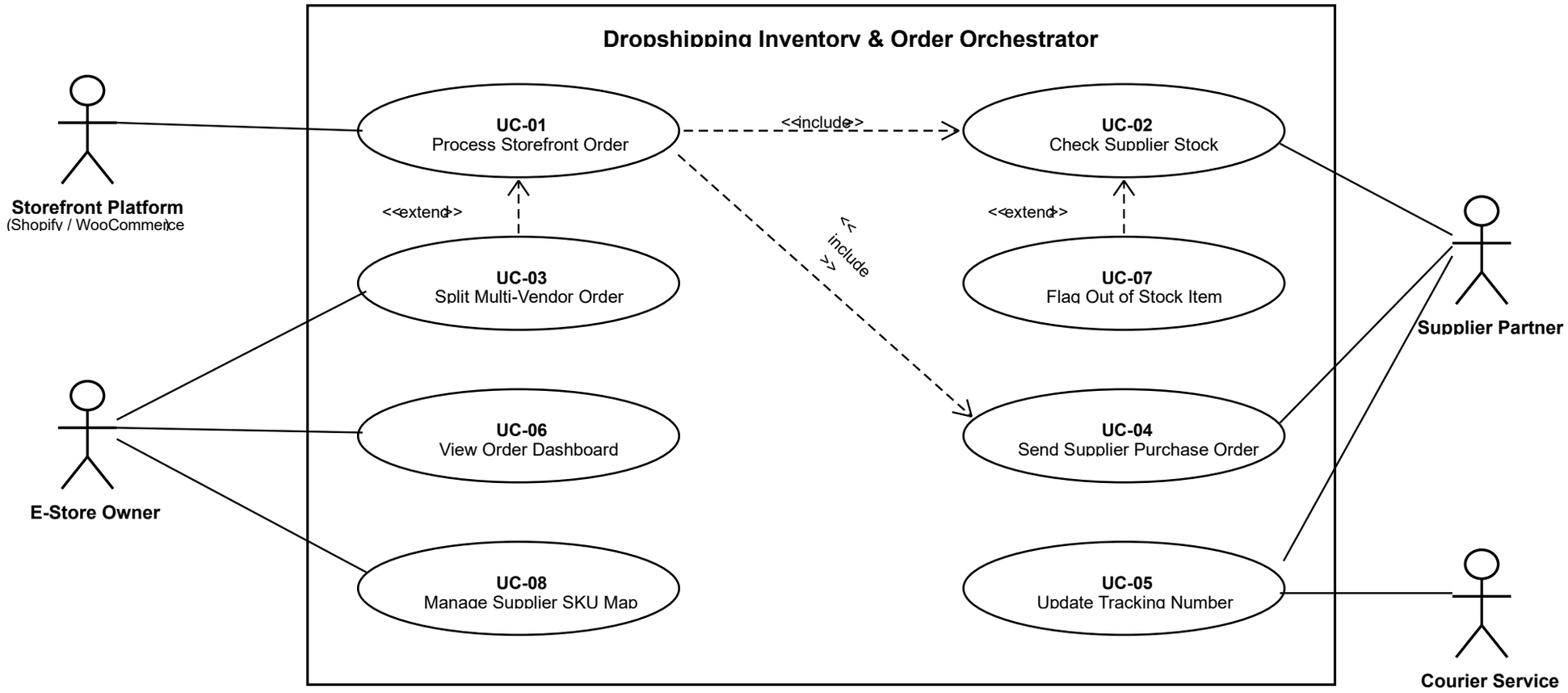
UC-05 Update Tracking Number (from FR-004)

UC-06 View Order Dashboard (from FR-005)

UC-07 Flag Out of Stock Item (from FR-003)

UC-08 Manage Supplier SKU Map (support for FR-002)

PART-2 Use-Case Diagram



Legend

Association: an actor takes part in the use case.

<<include>>: UC-01 always runs UC-02 and UC-04 as part of its own steps.

<<extend>>: UC-03 runs only if the order has more than one supplier.

UC-07 runs only if the supplier stock count is zero.

Part 3: Use-Case Flow Specification

Use Case ID	UC-01
Use Case Name	Process Storefront Order and Place Supplier Purchase Order
Primary Actor	Storefront Platform (Shopify / WooCommerce)
Secondary Actors	Supplier Partner, E-Store Owner
Goal	Turn one paid customer order into a purchase order for every supplier involved.
Related Requirements	FR-001, FR-002, FR-003, NFR-001, NFR-002

Preconditions

1. The E-Store Owner has connected the storefront and the webhook secret is saved in the system.
2. Every product/SKU in the store is linked to at least one supplier
3. The supplier API keys are valid and stored in the system
4. The customer has paid for the order on the storefront

Postconditions

1. There is one sub-order and one purchase order for each supplier involved in the order
2. Each purchase order holds the correct items and the customer delivery address
3. The order status on the dashboard is updated to “PO Sent.”
4. All the steps taken while processing the order are recorded in the audit log.

Main Success Scenario

1. The Storefront Platform sends an order webhook to the system.
2. The system checks the HMAC signature of the webhook and replies with HTTP 200 within 200 ms.
3. The system reads the order and saves the items, the quantities, and the delivery address.
4. The system finds the supplier for each SKU from the supplier SKU map.
5. The system calls the stock API of each supplier and reads the stock count [«include» UC-02 Check Supplier Stock] 6. Every item in the order is in stock
7. The system splits the order into one sub-order for each supplier. [«extend» UC-03 Split Multi-Vendor Order, runs only when the order has more than one supplier]
8. The system builds a purchase order for each sub-order with the items and the customer delivery address. [«include» UC-04 Send Supplier Purchase Order]
9. The system sends each purchase order to the matching Supplier Partner.
10. The supplier accepts the purchase order and returns a supplier order ID.
11. The system sets each sub-order status to PO Sent and stores the supplier order ID.
12. The system shows the updated order on the E-Store Owner dashboard.
13. The use case ends successfully.

Alternate Flow - 6a. Item is out of stock at the supplier

6a1. The supplier stock API returns a count of 0 for one or more items in the order.

6a2. The system does not build a purchase order for those items. [«extend» UC-07 Flag Out of Stock Item]

6a3. The system sets the affected sub-order status to On Hold.

6a4. The system sends an alert with the order ID and the SKU to the E-Store Owner within 1 minute.

6a5. The system checks the supplier SKU map for a backup supplier for that SKU.

6a6. If a backup supplier has stock, the flow goes back to step 7 with the backup supplier.

6a7. If no backup supplier has stock, the sub-order stays On Hold. The E-Store Owner then cancels or refunds the item from the dashboard, and the use case ends here